



# Exponent Limit Orderbook

Security Assessment

August 20th, 2025 — Prepared by OtterSec

---

Ajay Shankar Kunapareddy

[d1r3wolf@osec.io](mailto:d1r3wolf@osec.io)

---

Xiang Yin

[soreatu@osec.io](mailto:soreatu@osec.io)

---

Robert Chen

[r@osec.io](mailto:r@osec.io)

---

# Table of Contents

|                                                             |           |
|-------------------------------------------------------------|-----------|
| <b>Executive Summary</b>                                    | <b>3</b>  |
| Overview                                                    | 3         |
| Key Findings                                                | 3         |
| <b>Scope</b>                                                | <b>4</b>  |
| <b>Findings</b>                                             | <b>5</b>  |
| <b>Vulnerabilities</b>                                      | <b>6</b>  |
| OS-ELO-ADV-00   Missing Validation in Admin Fee Collection  | 9         |
| OS-ELO-ADV-01   Functionality Breakage After Vault Expiry   | 10        |
| OS-ELO-ADV-02   Precision Loss in Orderbook Balance         | 11        |
| OS-ELO-ADV-03   Stalled Counter-Offer Traversal             | 12        |
| OS-ELO-ADV-04   Failure to Remove Expired Offers            | 13        |
| OS-ELO-ADV-05   Failure to Update Amounts on Withdrawal     | 14        |
| OS-ELO-ADV-06   DOS via Frontrunning Account Creation       | 15        |
| OS-ELO-ADV-07   Improper Escrow Amount Conversion           | 16        |
| OS-ELO-ADV-08   Flawed Offer Handling Logic                 | 17        |
| OS-ELO-ADV-09   Incomplete Interest Collection Logic        | 18        |
| OS-ELO-ADV-10   Slippage Check Mismatch in Market Offers    | 19        |
| OS-ELO-ADV-11   Utilization of Incorrect YT Deposit Amount  | 20        |
| OS-ELO-ADV-12   Improper Assignment of Maker and Taker Fees | 21        |
| <b>General Findings</b>                                     | <b>22</b> |
| OS-ELO-SUG-00   Missing Validation Logic                    | 23        |
| OS-ELO-SUG-01   Code Refactoring                            | 25        |

TABLE OF CONTENTS

|                                           |    |
|-------------------------------------------|----|
| OS-ELO-SUG-02   Code Maturity             | 26 |
| OS-ELO-SUG-03   Inconsistency in Comments | 27 |
| OS-ELO-SUG-04   Unutilized/Redundant Code | 28 |

Appendices

|                            |    |
|----------------------------|----|
| Vulnerability Rating Scale | 30 |
| Procedure                  | 31 |

# 01 — Executive Summary

---

## Overview

Exponent Finance Labs engaged OtterSec to assess the **limit-orderbook** program. This assessment was conducted between August 6th and August 19th, 2025. A follow-up audit was performed between December 16th and December 20th, 2025. For more information on our auditing methodology, refer to [Appendix B](#).

## Key Findings

We produced 18 findings throughout this audit engagement.

In particular, we identified several high-risk vulnerabilities, including the lack of proper admin and escrow account validation in admin fee collection logic, allowing unauthorized fee withdrawals, and fails when attempting YT withdrawals from expired vaults, resulting in denial of service for fee collection ([OS-ELO-ADV-00](#)). Similarly, fund withdrawal also fails after vault expiry as the withdrawal logic calls YT withdrawal functions ([OS-ELO-ADV-01](#)). Also, usage of f64 in balance updates introduced rounding errors and state divergence from on-chain amounts, risking accounting inconsistencies ([OS-ELO-ADV-02](#)).

Additionally, when finding a counteroffer, the loop may stall if the next offer index points to an expired offer, preventing traversal to the next price node and resulting in skipping valid counteroffers ([OS-ELO-ADV-03](#)), and accumulated expired offers may be spammed to bloat the orderbook, degrading performance and eventually blocking new valid orders ([OS-ELO-ADV-04](#)).

Furthermore, we highlighted the possibility to pre-fund the predictable escrow PDA with lamports, resulting in the account creation function failing and blocking orderbook creation ([OS-ELO-ADV-06](#)).

We also made recommendations for updating the codebase for improved functionality and to mitigate potential security issues ([OS-ELO-SUG-01](#)), and suggested removing redundant and unutilized code instances ([OS-ELO-SUG-04](#)). Moreover, we advised adhering to coding best practices ([OS-ELO-SUG-02](#)) and including additional safety checks within the codebase to improve security ([OS-ELO-SUG-00](#)).

## 02 — Scope

---

The source code was delivered to us in a Git repository at <https://github.com/exponent-finance/exponent-core>. This audit was performed against [89675be](#).

The follow-up audit was performed against:

- 1. [9e6ee0d](#).
- 2. [dfe57c5](#).
- 3. [d40376c](#).
- 4. [e0bf0a5](#).
- 5. [4c9ebc2](#).
- 6. [c338e59](#).

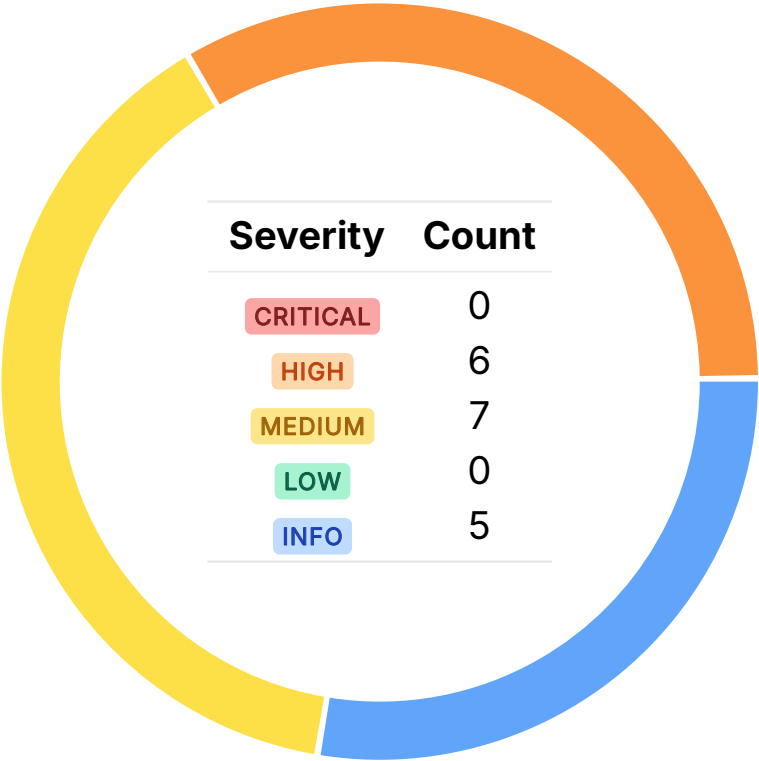
A brief description of the program is as follows:

| Name            | Description                                                                                                                                                                                                                                                                                                                                                                                                            |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| limit-orderbook | A fixed-rate orderbook is built on top of the core program's vault module, where offers are split into SellYt and BuyYt and quoted via the natural logarithm of the implied APY. Similar flash swap logic as in the Exponent core program is included to support virtual cases such as PT → SY and SY → PT. Offers are stored as a linked list, with head and tail pointers maintained in the corresponding PriceNode. |

# 03 — Findings

Overall, we reported 18 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



## 04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities we identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in [Appendix A](#).

| ID            | Severity | Status     | Description                                                                                                                                                                                                                                             |
|---------------|----------|------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| OS-ELO-ADV-00 | HIGH     | RESOLVED ✓ | The admin fee collection logic lacks proper admin and escrow account validation, allowing unauthorized fee withdrawals.                                                                                                                                 |
| OS-ELO-ADV-01 | HIGH     | RESOLVED ✓ | <code>collect_admin_fee</code> , <code>withdraw_funds</code> and <code>remove_offer</code> fail after vault expiry because they still call <code>YT</code> withdrawal functions that are invalid post-expiry, locking funds and blocking offer removal. |
| OS-ELO-ADV-02 | HIGH     | RESOLVED ✓ | The utilization of <code>f64</code> for balance updates while CPIs utilize precise <code>Number</code> ( <code>U256</code> ) results in rounding errors and state inconsistencies.                                                                      |
| OS-ELO-ADV-03 | HIGH     | RESOLVED ✓ | The loop in <code>find_counter_offer</code> may stall if <code>next_offer_idx</code> points to an expired offer, preventing traversal to the next price node and resulting in valid counter-offers to be skipped.                                       |
| OS-ELO-ADV-04 | HIGH     | RESOLVED ✓ | Accumulated expired offers may be spammed to bloat the <code>orderbook</code> , degrading performance and eventually blocking new valid orders.                                                                                                         |

|               |        |            |                                                                                                                                                                                                                                                                                                                                                                |
|---------------|--------|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| OS-ELO-ADV-05 | HIGH   | RESOLVED ✓ | <code>withdraw_funds</code> fails to reduce <code>staked_yt_amount</code> , letting users accrue interest on already-withdrawn <code>YT</code> , enabling perpetual interest claims. Also, orderbook financials are not updated with the withdraw amounts in <code>collect_admin_fee</code> , and the <code>SY</code> exchange rate is not properly validated. |
| OS-ELO-ADV-06 | MEDIUM | RESOLVED ✓ | It is possible to pre-fund the predictable escrow PDA with lamports, resulting in the account creation function failing and blocking orderbook creation.                                                                                                                                                                                                       |
| OS-ELO-ADV-07 | MEDIUM | RESOLVED ✓ | <code>get_amounts_on_filling_escrow</code> wrongly reconverts <code>sended_amount</code> (already in <code>YT</code> units) as if it were <code>SY</code> , resulting in double conversion.                                                                                                                                                                    |
| OS-ELO-ADV-08 | MEDIUM | RESOLVED ✓ | <code>fill_offer</code> utilizes the wrong variable for balance updates, risking fund mismanagement.                                                                                                                                                                                                                                                           |
| OS-ELO-ADV-09 | MEDIUM | RESOLVED ✓ | <code>collect_admin_fee</code> instruction records interest locally from <code>yt_fee_balance</code> but fails to invoke <code>do_cpi_collect_interest</code> , resulting in unbacked fee accrual and mismatch with the core vault state.                                                                                                                      |
| OS-ELO-ADV-10 | MEDIUM | RESOLVED ✓ | The <code>market_offer</code> instruction validates slip-page against <code>total_filled_amount</code> , which does not reflect the actual asset received in virtual flows and ignores fee deductions, potentially allowing trades to proceed below the intended minimum output.                                                                               |
| OS-ELO-ADV-11 | MEDIUM | RESOLVED ✓ | <code>wrapper_market_offer</code> deposits the requested input amount instead of the actual <code>YT</code> received from the trade.                                                                                                                                                                                                                           |



---

|               |        |            |                                                                                                              |
|---------------|--------|------------|--------------------------------------------------------------------------------------------------------------|
| OS-ELO-ADV-12 | MEDIUM | RESOLVED ✓ | The maker and taker fees are assigned from the wrong escrow results, effectively swapping their attribution. |
|---------------|--------|------------|--------------------------------------------------------------------------------------------------------------|

---

## Missing Validation in Admin Fee Collection HIGH

OS-ELO-ADV-00

### Description

In `collect_admin_fee` logic, several critical validations are missing, leading to unintended token transfers. In particular, the absence of a check to ensure the transaction's signer matches `orderbook.admin` allows any arbitrary signer to invoke the instruction, impersonate the admin, and collect fees, resulting in unauthorized transfer of protocol funds. Additionally, the instruction fails to verify that `token_escrow_yt` and `token_escrow_pt` match the corresponding fields in the `orderbook` structure, introducing the risk of unauthorized token deductions.

### Remediation

Verify the signer `admin` matches the `orderbook.admin` and that `token_escrow_yt` and `token_escrow_pt` correspond to respective fields in the `orderbook` structure.

### Patch

Resolved in [PR#2678](#).

## Functionality Breakage After Vault Expiry HIGH

OS-ELO-ADV-01

### Description

`collect_admin_fee` and `withdraw_funds` currently always call `do_cpi_withdraw_yt` and `do_transfer_yt_out`, regardless of whether the vault is expired. However, once the vault expires, these CPI calls will fail because `YT` withdrawals are no longer valid, causing the entire transaction to revert. As a result, the admin is unable to collect fees, and users cannot withdraw other eligible funds once the vault has expired. Similarly, the `do_cpi_withdraw_yt` call in `remove_offer` prevents outstanding offers from being cleared post-expiry.

```
>_ orderbook/withdraw_funds.rs
```

RUST

```
pub fn handler<'info>([...]) -> Result<WithdrawFundsEvent> {
    [...]
    do_cpi_withdraw_yt(
        &ctx.accounts.address_lookup_table,
        &ctx.accounts.cpi_account_orderbook.exponent_core_cpi_accounts,
        &ctx.accounts.cpi_account_orderbook.sy_cpi_accounts,
        ctx.remaining_accounts,
        &ctx.accounts.to_account_infos(),
        *ctx.accounts.exponent_core.key,
        yt_amount,
        &[&signer_seeds],
    )?;

    ctx.accounts.do_transfer_sy_out(sy_amount, &[&signer_seeds]).unwrap();
    ctx.accounts.do_transfer_yt_out(yt_amount, &[&signer_seeds]).unwrap();
    ctx.accounts.do_transfer_pt_out(pt_amount, &[&signer_seeds]).unwrap();
    [...]
}
```

### Remediation

Skip calling `do_cpi_withdraw_yt` and `do_transfer_yt_out` in `collect_admin_fee` and `withdraw_funds` if the vault has expired. Also, skip invoking `do_cpi_withdraw_yt` in `remove_offer` to allow safe clearing of all offers after expiration.

### Patch

Resolved in [PR#2678](#) and [PR#2702](#).

## Precision Loss in Orderbook Balance HIGH

OS-ELO-ADV-02

### Description

The issue arises because `orderbook::update_on_strip_and_merge` updates `orderbook_raw.financials.pt_balance`, `orderbook_raw.financials.yt_balance`, and `orderbook_raw.financials.sy_balance` utilizing `f64`, while the actual CPIs (cross program invocations) in `do_strip_and_merge` (`merge` / `strip`) utilize `Number` (`U256` fixed-point). Since `f64` only has 53 bits of precision, balances with more significant bits lose accuracy, resulting in divergence between in-memory orderbook balances and the true on-chain amounts from CPIs.

### Remediation

Ensure that `update_on_strip_and_merge` is integrated into `do_strip_and_merge`, balances are updated directly from CPI results after the merge, and all `f64` utilization is replaced with `Number`.

### Patch

Resolved in [PR#2703](#).

## Stalled Counter - Offer Traversal HIGH

OS-ELO-ADV-03

### Description

The vulnerability occurs in the loop within `orderbook::find_counter_offer`, when the second-to-last offer in a `price_node` is valid but its `next_offer` (the last one) is already expired. After matching, `filled_offers.next_offer_idx` is set to this expired entry. On the next invocation, because `next_offer_idx != SENTINEL`, the code does not advance to the next `price_node` and gets stuck evaluating the expired offer repeatedly. This stalls the loop and prevents traversal into subsequent price nodes. As a result, valid offers at better prices may be skipped, leaving liquidity unmatched and potentially blocking order execution.

### Remediation

Advance `filled_offers.next_offer_idx` to the next register in the linked list when an expired offer is encountered.

### Patch

Resolved in [PR#2704](#).

## Failure to Remove Expired Offers HIGH

OS-ELO-ADV-04

### Description

Expired offers are currently skipped but never removed, resulting in them accumulating in the **orderbook**. An attacker may exploit this by spamming many short-expiry offers, quickly filling storage with expired entries. As more of these expired offers pile up, it bloats the **orderbook** and forces the matching engine to traverse through numerous invalid offers, degrading performance and increasing latency in finding valid matches. Over time, the buildup may exhaust the offer capacity, preventing the placement of new valid orders

### Remediation

Prune expired offers during matching to mitigate this issue.

### Patch

Resolved in [PR#2699](#).

## Failure to Update Amounts on Withdrawal HIGH

OS-ELO-ADV-05

### Description

In `withdraw_funds`, the `staked_yt_amount` is not reduced when users withdraw `YT`, leaving the escrow state inconsistent. This implies the program still considers the withdrawn `YT` as staked, allowing users to continue accruing interest on funds they no longer hold. Effectively, users may exploit this to claim perpetual interest. Over time, such abuse may drain the orderbook's reserves.

Similarly, in `collect_admin_fee`, when `SY` / `PT` / `YT` tokens are withdrawn, the orderbook's financial tracking variables are not updated, leaving inflated recorded balances that no longer match on-chain reality. This mismatch may result in incorrect interest accruals, solvency checks, or liquidity calculations. Additionally, the `sy_exchange_rate` is fetched directly without utilizing `validate_sy_exchange_rate`, which is responsible for handling vault expiry cases.

### Remediation

Reduce `staked_yt_amount` proportionally during withdrawals in `withdraw_funds`. Also, ensure orderbook financials are updated with the withdraw amounts in `collect_admin_fee`, and that `sy_exchange_rate` is sourced from `validate_sy_exchange_rate`.

### Patch

Resolved in [PR#2717](#) and [PR#2719](#).

## DOS via Frontrunning Account Creation MEDIUM

OS-ELO-ADV-06

### Description

Within `Orderbook_init::create_orderbook_token_account`, while creating an orderbook, `create_token_account` calls `system_program::create_account`, which fails if the target account already has lamports. Since the escrow PDAs are deterministic, an attacker may precompute their addresses and front-run the initialization by transferring a minimal lamport amount to them. Thus, when the legitimate program later tries to create the account, the system program detects it as *“already in use”* and aborts.

```
>_ orderbook/admin/orderbook_init.rs
```

RUST

```
fn create_account([...] -> Result<(), InstructionError> {  
    // if it looks like the `to` account is already in use, bail  
    {  
        let mut to = instruction_context  
            .try_borrow_instruction_account(transaction_context, to_account_index)?;  
        if to.get_lamports() > 0 {  
            ic_msg!(  
                invoke_context,  
                "Create Account: account {:?} already in use",  
                to_address  
            );  
            return Err(SystemError::AccountAlreadyInUse.into());  
        }  
        allocate_and_assign(&mut to, to_address, space, owner, signers, invoke_context)?;  
    }  
    [...]  
}
```

The issue exists both in the `create_orderbook_token_account` and `make_orderbook_account` functions.

### Remediation

Empty the balance of the account (if it is not zero) before calling `system_program::create_account`, or manually create the account by invoking `transfer`, `allocate`, and `assign`, similar to what anchor does.

### Patch

Resolved in [PR#2674](#) and [PR#2701](#).



## Improper Escrow Amount Conversion MEDIUM

OS-ELO-ADV-07

### Description

The vulnerability in `RawOrderbook::get_amounts_on_filling_escrow` arises because `sended_amount`, which already represents the filled `YT` amount from the counteroffer, is incorrectly passed into `calculate_base_amount_from_sy`. This results in a double conversion, as the amount is treated as if it were `SY` and converted again utilizing the exchange rate. Such a miscalculation results in an incorrect `added_amount` credited to escrow and distorts fee computation.

```
>_ src/state/orderbook.rs
```

RUST

```
pub fn get_amounts_on_filling_escrow([...]) -> FillEscrowResult {
    let total_to_add_amount = if offer_type == OfferType::BuyYt {
        [...]
    } else {
        if virtual_offer {
            calculate_base_amount_from_sy(sended_amount, sy_exchange_rate)
        } else {
            filled_amount
        }
    };
    [...]
}
```

### Remediation

Refrain from passing the `sended_amount` into `calculate_base_amount_from_sy`.

### Patch

Resolved in [PR#2716](#).

## Flawed Offer Handling Logic MEDIUM

OS-ELO-ADV-08

### Description

`fill_offer` incorrectly utilizes `virtual_offer` in the `add_balance_on_fill_offer` call on the taker escrow, instead of `incoming_virtual_offer`, which actually represents the taker's virtual status. This results in the taker's balance updates to be applied utilizing the counterparty's configuration rather than its own. Thus, real takers may be treated as virtual or vice versa, resulting in incorrect fee application and balance accounting.

```
>_ src/state/orderbook.rs
```

RUST

```
pub fn fill_offer([...]) -> Result<StripAndMergeAmount> {  
    [...]  
    self.add_balance_on_fill_offer(  
        taker_user_escrow_idx,  
        virtual_offer,  
        offer_type.counter_type(),  
        taker_fill_escrow_result,  
        sy_exchange_rate_number,  
    )  
    [...]  
}
```

### Remediation

Ensure the call to `add_balance_on_fill_offer` on the taker escrow utilizes `incoming_virtual_offer` instead of `virtual_offer`.

### Patch

Resolved in [PR#2718](#).

## Incomplete Interest Collection Logic MEDIUM

OS-ELO-ADV-09

### Description

`collect_admin_fee` withdraws interest derived from `yt_fee_balance` by moving it into `staged_sy`, but it does so without calling `do_cpi_collect_interest` on the core program. Thus, the protocol only updates its local accounting, without actually collecting interest from the underlying vault. As a result, the admin may withdraw unbacked staged yield, creating a mismatch between the vault's real balances and the `orderbook` state.

### Remediation

Ensure to collect interest from the core program via `do_cpi_collect_interest` when collecting interest derived from `yt_fee_balance`.

### Patch

Resolved in [PR#2720](#).

## Slippage Check Mismatch in Market Offers

MEDIUM

OS-ELO-ADV-10

### Description

In `market_offer`, the slippage check is performed by comparing `total_filled_amount` against `min_amount_out`. However, `total_filled_amount` does not necessarily represent the final asset amount the taker receives:

1. For `BuyYt` with `virtual_offer`, the actual output is `PT→SY`, but the check is done on `YT`.
2. For `Sellyt` with `virtual_offer`, the actual output is `SY→PT`, but the check is done on `SY`.

```
>_ src/state/orderbook.rs RUST  
  
pub fn market_offer([...]) -> Result<PostOfferOutput> {  
    [...]  
    filled_offers  
        .total_filled_amount  
        .checked_sub(min_amount_out)  
        .expect("min amount out has not been reached");  
    [...]  
}
```

Additionally, fees deducted during settlement are not considered in the slippage validation. As a result, the taker may receive less than `min_amount_out`, violating user expectations and exposing them to unfavorable execution.

### Remediation

Perform slippage validation against the final post-fee output amounts in the correct asset units (SY or PT), depending on the offer type and whether the flow is virtual. Ensure that fee deductions are factored into the slippage comparison.

### Patch

Resolved in [PR#2754](#).

## Utilization of Incorrect YT Deposit Amount MEDIUM

OS-ELO-ADV-11

### Description

`wrapper_market_offer::handler` deposits amount into `cpi_user_deposit_yt` instead of the actual executed output `withdraw_funds_result.amount_yt_out`. If the order is partially filled or fees are applied, this over-credits `YT` relative to what the `orderbook` released.

```
>_ src/instructions/wrappers/wrapper_market_offer.rs
```

RUST

```
pub fn handler<'info>([...]) -> Result<WrapperMarketOfferEvent> {  
    [...]  
    if withdraw_funds_result.amount_yt_out > 0 {  
        core_cpi::cpi_user_deposit_yt(  
            *ctx.accounts.exponent_core.key,  
            redeem_sy_or_deposit_yt_accounts,  
            redeem_sy_or_deposit_yt_accounts.to_vec()  
                .to_account metas(None),  
            amount,  
        )?;  
    }  
    [...]  
}
```

### Remediation

Utilize `withdraw_funds_result.amount_yt_out` as the `amount` for `cpi_user_deposit_yt`.

### Patch

Resolved in [PR#3376](#).

## Improper Assignment of Maker and Taker Fees

MEDIUM

OS-ELO-ADV-12

### Description

In `RawOrderbook::fill_offer`, `taker_fill_escrow_result` represents fees charged to the taker, while `seller_fill_escrow_result` represents fees charged to the maker. However, the code assigns `maker_fee_amount` from `taker_fill_escrow_result.fee_amount` and `taker_fee_amount` from `seller_fill_escrow_result.fee_amount`, effectively swapping their roles. As a result, any downstream logic that relies on `strip_and_merge_amounts` will misinterpret who actually paid which fee.

```
>_ src/state/orderbook.rs
```

RUST

```
pub fn fill_offer([...]) -> Result<StripAndMergeAmount> {  
    [...]  
    strip_and_merge_amounts.maker_fee_amount = taker_fill_escrow_result.fee_amount;  
    strip_and_merge_amounts.taker_fee_amount = seller_fill_escrow_result.fee_amount;  
    [...]  
}
```

### Remediation

Ensure to correctly assign the `maker_fee_amount` and `taker_fee_amount` values in `fill_offer`.

### Patch

Resolved in commit [8e38289](#).

# 05 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

| ID            | Description                                                                                                                        |
|---------------|------------------------------------------------------------------------------------------------------------------------------------|
| OS-ELO-SUG-00 | There are several instances where proper validation is not performed, resulting in potential issues.                               |
| OS-ELO-SUG-01 | Recommendation for refactoring the orderbook initialization logic to improve functionality and mitigate potential security issues. |
| OS-ELO-SUG-02 | Suggestions regarding inconsistencies in the codebase and ensuring adherence to coding best practices.                             |
| OS-ELO-SUG-03 | There are multiple instances of inaccurate comments containing incorrect descriptions or typographical errors.                     |
| OS-ELO-SUG-04 | The codebase contains multiple cases of redundant or unutilized code that should be removed.                                       |

## Missing Validation Logic

OS-ELO-SUG-00

### Description

1. In both `post_offer` and `market_offer` in `RawOrderbook`, utilizing `MAX_FILLED_OFFERS >= filled_counter` allows the loop to process one extra counter offer beyond the intended maximum, violating orderbook constraints. Change the check to `MAX_FILLED_OFFERS > filled_counter` to ensure strict enforcement of the maximum fill limit.

`>_ src/state/orderbook.rs`

RUST

```
pub fn market_offer([...]) -> Result<PostOfferOutput> {
    [...]
    {
        let price_iter = &mut self.get_price_iterator(offer_type).peekable();
        while remaining_amount > self.configuration_options.threshold_amount
            && MAX_FILLED_OFFERS >= filled_counter
        {
            [...]
        }
        [...]
    }
}
```

2. In `RawOrderbook::find_prev_offer_idx`, checking `current_offer_idx == offer_idx` assumes the price node always contains offers. If `current_offer_idx` is actually `SENTINEL` (no offers), calling `get_register` on it may result in arithmetic errors. The safer approach is to first check `current_offer_idx == SENTINEL` before comparing with `offer_idx`, preventing potential runtime errors.

`>_ src/state/orderbook.rs`

RUST

```
pub fn find_prev_offer_idx(&mut self, offer_idx: u32) -> Result<u32> {
    [...]
    if current_offer_idx == offer_idx {
        return Ok(SENTINEL);
    }
    while current_offer_idx != offer_idx {
        prev_offer_idx = current_offer_idx;
        current_offer_idx = self
            .offers
            .get_register(current_offer_idx, NEXT_OFFER_REGISTER);
        if current_offer_idx == SENTINEL {
            return Err(ExponentOrderbookError::IncorrectPriceNodeForOfferIdx.into());
        }
    }
    return Ok(prev_offer_idx);
}
```



3. If `sec_remaining` is zero, `calculate_price_in_assets_by_apy_pt` produces a zero price, and the fee rate becomes zero, resulting in arithmetic errors in `get_fee_from_total`. Verify `sec_remaining != 0` before offer matching.
4. Both `wrapper_collect_admin_fee` and `wrapper_withdraw_funds` always call `redeem_sy`, even when `sy_amount` is zero. Adding a guard ( `sy_amount > 0` ) will avoid unnecessary CPI calls, saving compute.

## Remediation

Add the validations stated above.

## Patch

Resolved in [PR#2715](#).

## Code Refactoring

OS-ELO-SUG-01

### Description

1. Utilizing `remaining_accounts` for `cpi_init_yield_token_position` in `initialize_yield_position` bypasses all account validation checks and may allow an unintended `yield_position` to be initialized. Utilize existing accounts for `cpi_init_yield_token_position`, which are validated.

```
>_ orderbook/admin/orderbook_init.rs
```

RUST

```
fn initialize_yield_position(  
    &self,  
    rem_accounts: &[AccountInfo],  
    seeds: &[&[u8]],  
) -> Result<()> {  
    cpi_init_yield_token_position(self.exponent_core.key(), rem_accounts, seeds)  
}
```

2. Currently, transferring a fixed lamport amount to `yield_position` may overfund the account if it already holds some lamports. Instead, utilize `lamports_to_transfer.saturating_sub(ctx.accounts.yield_position.lamports())` to ensure only the additional required lamports are sent.

### Remediation

Incorporate the above refactors.

### Patch

Resolved in [PR#2676](#)

## Code Maturity

OS-ELO-SUG-02

### Description

1. Since the `yield_position` account's lamports are increased, it must be marked as `mut` to reflect that its state will change during the instruction execution.
2. In `post_offer`, the current check with `checked_sub` enforces `post_amount ≥ threshold_amount`, allowing equality, which violates the intended rule that `post_amount` must be strictly greater. Explicitly check `post_amount > threshold_amount` to ensure correctness.

```
>_ src/instructions/orderbook/post_offer.rs
```

RUST

```
pub fn validate_accounts(&self, post_amount: u64) -> Result<()> {  
    [...]  
    let raw: &RawOrderbook = bytemuck::from_bytes(&data[8..]);  
    post_amount.checked_sub(raw.configuration_options.threshold_amount).expect("Post  
        ↳ amount must be greater than threshold amount");  
    [...]  
}
```

3. In `market_offer::do_strip_and_merge`, the base amount for depositing `YT` ( `base_diff_strip_amount` ) is recalculated from the stripped `SY` amount and the current exchange rate. A more precise approach is to utilize the `amount_py_out` from the `StripEvent`, which reflects the exact base amount produced.

### Remediation

Implement the above suggestions.

### Patch

1. Issues #1 was resolved in [PR#2676](#).
2. Issues #2 was resolved in [PR#2698](#).
3. Issues #3 was resolved in [PR#2703](#).

## Inconsistency in Comments

OS-ELO-SUG-03

### Description

The comment for the `orderbook` account has a typographical error where *multiply orderbooks* is written instead of *multiple orderbooks*. Also, several inaccurate comments incorrectly describe the structure field or account as listed below:

- The comment for `UserEscrow.user` incorrectly describes the field.
- The comments for `OrderbookInit.token_escrow_sy` and `OrderbookInit.token_escrow_yt` incorrectly describe their respective fields.
- The comments for `token_escrow_sy` and `token_escrow_pt` incorrectly describe their respective accounts in most CPI Accounts.

### Remediation

Ensure that comments in the referenced areas are consistent with the code implementation and provide an accurate definition.

### Patch

Resolved in [PR#2700](#).

## Unutilized/Redundant Code

OS-ELO-SUG-04

### Description

1. `util::signer_seeds_vec_to_slice` seems redundant since its logic may be folded into `RawOrderbook::signer_seeds`. Making `signer_seeds` return slices directly reduces duplication.

```
>_ orderbook/util.rs RUST  
  
pub fn signer_seeds_vec_to_slice(signer_seeds_vec: &[Vec<u8>]) -> [&[u8]; 4] {  
    [  
        &signer_seeds_vec[0],  
        &signer_seeds_vec[1],  
        &signer_seeds_vec[2],  
        &signer_seeds_vec[3],  
    ]  
}
```

2. The `payer` field in `SetConfigurationOptions` is unutilized and may be removed. Also, `user_escrow::has_yt_for_interest` is not utilized and may be removed.
3. The utilization of `enumerate` in `withdraw_funds`, `collect_interest`, and `get_or_create_user_escrow_idx` is redundant because `.position()` already returns the matching index.
4. In many of the CPI accounts, `cpi_account_orderbook` and `address_lookup_table` are marked as `mut` even though they are not actually written to or modified during execution. If these accounts are only read from, there is no requirement to mark them as `mut`.
5. In `wrapper_withdraw_funds`, the `yield_position` is unutilized. Also, `cpi_withdraw_user_yt` is unutilized in `core_cpi`.
6. In `collect_admin_emission`, emissions are first sent to the orderbook's escrow and then transferred again to the admin account, adding an unnecessary extra step. Thus, `do_transfer_out` is redundant and `token_emission_adminbe` may be passed directly as `emission_dst` for `collect_emission`.

### Remediation

Remove the redundant and unutilized code instances.

## Patch

1. Issue #1 was resolved in [PR#2714](#).
2. Issue #2 was resolved in [PR#2675](#) and [PR#2697](#).
3. Issue #3 was resolved in [PR#2697](#).
4. Issue #4 was resolved in [PR#2697](#).
5. Issue #5 was resolved in [PR#2714](#).
6. Issue #6 was resolved in [PR#2714](#).

# A — Vulnerability Rating Scale

---

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the [General Findings](#).

---

## CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
  - Improperly designed economic incentives leading to loss of funds.
- 

## HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
  - Exploitation involving high capital requirement with respect to payout.
- 

## MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
  - Forced exceptions in the normal user flow.
- 

## LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.
- 

## INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
  - Improved input validation.
-

## B — Procedure

---

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.